ASSIGNMENT 5 – CO5

Reliable Software with Runtime Polymorphism and Exceptions

Case Study: Hospital Billing and Insurance Claims

Name: TELAPROLU ROHITA SAI

Register No: 192425041

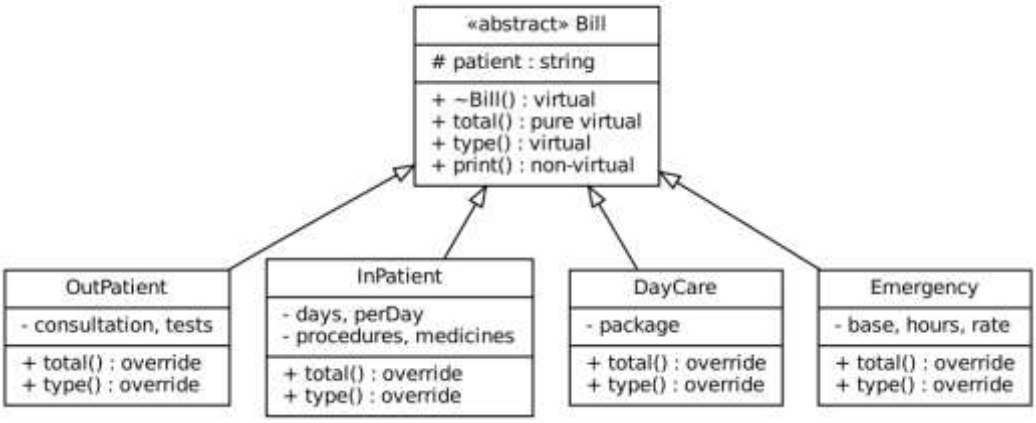
PART A – DESIGN

Task A1 – Polymorphic Bill Hierarchy

A1.1 Common interface

Every bill must provide total() (amount payable, differs per kind), type() (short code such as OP, IP, DC, ER) and print(). Common data: the patient name/record id. Everything else (consultation, room rate, package price, hours) belongs to the derived class.

A1.2 Hierarchy (UML)



Virtual: ~Bill() and type() (default "Bill"). Pure virtual: total(). Overridden: total() and type() in every derived class. print() is non-virtual and calls the virtual functions.

A1.3 Pseudocode – day's total

```
ALGORITHM DayTotal(bills[0..n-1])
BEGIN
    sum ← 0
    FOR i ← 0 TO n-1 DO
        sum ← sum + bills[i]->total()      // late binding picks the right rule
    END FOR
    RETURN sum
END
```

Alternative Concept and Complexity Analysis

One Bill structure with a type field and a switch in total():

```
ALGORITHM Total(b)
BEGIN
    SWITCH b.type
        CASE OP: RETURN b.consultation + b.tests
        CASE IP: RETURN b.days * b.perDay + b.procedures + b.medicines
        CASE DC: RETURN b.package
        CASE ER: RETURN b.base + b.hours * b.rate
    END SWITCH
END
```

Measure	Abstract base + virtual	Single struct + switch

Time to total n bills	O(n): one virtual call per bill	O(n): one switch per bill
Measured (10,000,000 bills, -O2)	95.3 ms	87.8 ms
Space per bill (measured)	OutPatient 56, InPatient 72, DayCare 48, Emergency 64 bytes (includes vptr)	72 bytes for every bill (all fields present)
Code changed for a new type	Add one new class; no existing code edited	Edit the struct, the type enum and every switch (total, print ...)

Task A2 – Runtime Binding

A2.1 Early and late binding

With Bill* b = new OPBill; the call b->total() is checked at compile time against the static type Bill (the name must exist there). If total() is non-virtual the compiler binds the call to Bill::total() immediately (early binding). If it is virtual the compiler emits code that reads the object's vptr at run time and calls the function stored in that class's vtable slot (late binding), so the dynamic type (OPBill) decides.

A2.2 Virtual tables

Slot	Bill (abstract)	OutPatient	InPatient
[0] destructor	Bill::~Bill	OutPatient::~OutPatient	InPatient::~InPatient
[1] total()	pure virtual (no body)	OutPatient::total	InPatient::total
[2] type()	Bill::type	OutPatient::type	InPatient::type

Every object of a class with virtual functions starts with a hidden vptr pointing to its class's vtable. b->total() runs: (1) load the object's vptr, (2) index slot [1], (3) jump to that function. For an OutPatient object this reaches OutPatient::total, for an InPatient object InPatient::total.

```
b ──► [ vptr | patient | consultation | tests ]  (OutPatient object)
            │
            └──► vtable(OutPatient): [~OutPatient][OutPatient::total][OutPatient::type]
                                                    ▲ b->total() calls slot [1]
```

A2.3 Virtual destructor

delete b on a Bill* must run the destructor of the real object. If ~Bill() is not virtual, only Bill's destructor runs: the derived part is never destroyed (leaks, files not closed) and behaviour is undefined. A virtual destructor makes delete use the vtable, so ~OutPatient runs first and then ~Bill.

Predict the Output

My predicted output: 500 ~OPBill ~Bill

```
#include <iostream>
struct Bill {
    virtual ~Bill() { std::cout << "~Bill "; }
    virtual double total() const = 0;
    void show() const { std::cout << total() << ' '; }
};
struct OPBill : Bill {
    ~OPBill() override { std::cout << "~OPBill "; }
    double total() const override { return 500; }
};
int main() { Bill* b = new OPBill; b->show(); delete b; std::cout << '\n'; }
```

Output:

```
500 ~OPBill ~Bill
```

Matches. show() is non-virtual, but inside it total() is called on *this whose static type is Bill, and total() is virtual, so the call is dispatched through the vptr to OPBill::total() (500). delete b calls ~OPBill first, then ~Bill (virtual destructor).

With virtual removed from ~Bill: predicted and actual output is 500 ~Bill (only the base destructor runs; undefined behaviour). g++ warns:

```
warning: deleting object of abstract class type 'Bill' which has non-virtual destructor will cause
undefined behavior [-Wdelete-non-virtual-dtor]
```

AddressSanitizer reports nothing for this tiny program because OPBill has no data members, so the base and derived objects have the same size (8 bytes). The defect shows up when the derived class is bigger, as in Task B1.4 (new-delete-type-mismatch, 56 bytes allocated, 40 bytes deleted).

Alternative Concept and Complexity Analysis

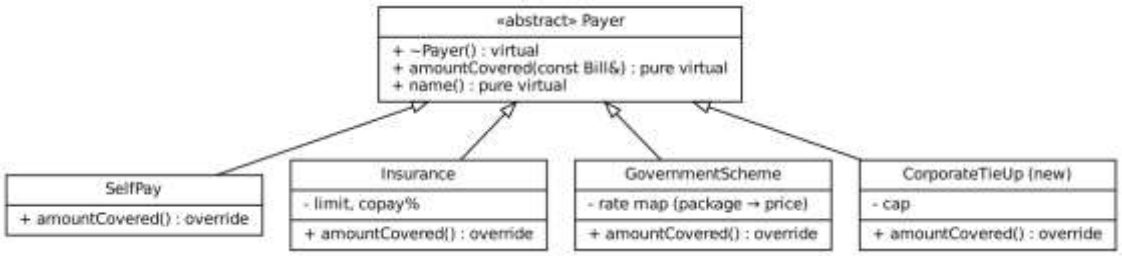
total() is not virtual and the program tests the type of each bill with dynamic_cast:

```
ALGORITHM Total(b)
BEGIN
    IF b is OutPatient THEN RETURN OutPatient part of b -> total()
    ELSE IF b is InPatient THEN RETURN ...
    ELSE IF b is DayCare ... ELSE IF b is Emergency ...
END
```

Measure	Virtual call	dynamic_cast chain (k types)
Time per bill	O(1): one vtable lookup	O(k): up to k casts (each compares type information)
Measured (10,000,000 bills, k = 4)	95.3 ms	306.1 ms (about 3.2 times slower)
Space	one 8-byte vptr per object	one vptr per object as well (dynamic_cast also needs a polymorphic base), no space saved
New bill type	add class	add class and edit the chain

Task A3 – Abstract Classes (Payer)

A3.1 Payer interface




```
ALGORITHM Settle(bill, payer)
BEGIN
    covered ← payer->amountCovered(bill)         // pure virtual, per payer rule
    patientPays ← bill->total() - covered
    RETURN patientPays
END

amountCovered rules:
    SelfPay          : RETURN 0
    Insurance        : IF total > limit THEN THROW ClaimError("exceeds cover")
                       RETURN total * (100 - copay) / 100
    GovernmentScheme : IF package not in rate table THEN THROW PackageError
                       RETURN MIN(rate[package], total)
```

A3.2 Extension without changing existing classes

A corporate tie-up is a new class CorporateTieUp derived from Payer that overrides amountCovered() (covered = MIN(cap, total)). Bill, Payer, SelfPay, Insurance, GovernmentScheme and the settlement code that works through Payer* are not edited; only the place that creates the payer object names the new class. It is implemented and tested in Task B4.

Alternative Concept and Complexity Analysis

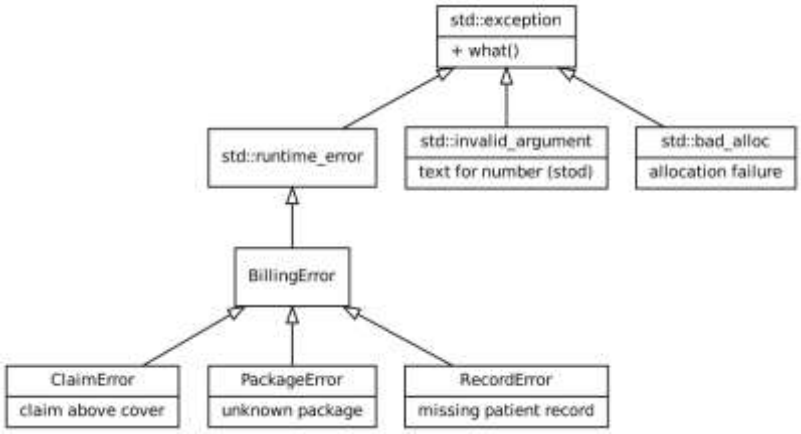
```
ALGORITHM MakeInsurance(limit, copay)
BEGIN
    RETURN function(bill): IF bill.total > limit THEN THROW ClaimError
                           RETURN bill.total * (100 - copay) / 100
END                       // limit and copay are captured inside the function object
```

Measure	Abstract Payer class	std::function / function pointer
Time per call	O(1) (virtual call)	O(1) (indirect call; std::function adds a small wrapper cost)
Space per payer (measured)	Insurance 24, SelfPay 8, GovScheme 56 bytes	std::function 32 bytes (+ heap if the captured data is large); plain function pointer 8 bytes
Where data such as cover limit lives	Private data members of the derived class	In the closure (captured variables) or global/extra argument; a plain function pointer cannot hold it
Multiple operations (name, amountCovered)	One class keeps them together	Need one callable per operation

Task A4 – Exception Strategy

A4.1 Errors and exception classes

Error	Exception	Thrown by
Negative charges	BillingError ("negative charge")	Bill constructors
Claim above cover	ClaimError : BillingError	Insurance::amountCovered
Unknown package	PackageError : BillingError	GovScheme::amountCovered
Missing patient record	RecordError : BillingError	Bill constructor (empty patient)
Text where a number is expected	std::invalid_argument	std::stod
Memory allocation failure	std::bad_alloc	operator new



A4.2 Exception specifications

- May throw: Bill constructors (validation), amountCovered() of Insurance and GovScheme, any function that allocates or parses text.
- Must never throw (noexcept): destructors (implicitly noexcept), total() and type() of an already-built bill (they only do arithmetic), name(), and swap/move operations.
- Why destructors must not throw: during stack unwinding destructors run because of an active exception; if one throws a second exception, the program calls std::terminate and all recovery is lost. (Old throw() specifications were removed in C++17; noexcept is the specification used.)

Alternative Concept and Complexity Analysis

```
ALGORITHM Approve(bill, cover, OUT result)     // returns a Status instead of throwing
BEGIN
    IF bill < 0 THEN RETURN NEGATIVE
    IF bill > cover THEN RETURN EXCEEDS_COVER
    result ← bill ;  RETURN OK
END
caller: status ← Approve(...) ; IF status ≠ OK THEN handle / return status upward
```

Measure (1,000,000 bills, measured)	Return codes	Exceptions
Normal path, 0% invalid	0.61 ms	2.11 ms (small absolute difference: 1.5 ms per million bills)
Error path, 1% invalid	0.57 ms	8.64 ms
Error path, 50% invalid	3.92 ms	339.80 ms (about 87 times slower)
Extra space and code	every function returns a status or extra OUT parameter; every caller must test and forward it	none on the normal path; one catch where the error is handled

Task A5 – Recovery and Rethrowing

A5.1 Where to catch

Layer	Action
Bill constructors, Payer::amountCovered	Detect and throw (BillingError, ClaimError, PackageError, RecordError)
Settlement function (middle layer)	Catch ClaimError only, write an audit log line, then rethrow with throw; (same object, same type)
Billing loop (per bill)	Catch BillingError: skip that bill, mark it rejected with the reason, keep processing the other bills
main (top level)	Catch std::exception and catch(...): report to the user and return a non-zero code; nothing is lost because bills are kept in RAII containers

A5.2 No leaks on errors

When an exception is thrown, C++ performs stack unwinding: it leaves each function in turn and destroys all fully constructed local objects in reverse order of construction until a matching catch is found. If memory and files are owned by objects with destructors (std::vector, std::unique_ptr, std::string, an ofstream, or a user-written Guard), their destructors run during unwinding and release the resources, so no explicit cleanup code is needed on the error path.

Predict the Output

My predicted output: ~f ~main caught 42

```
#include <iostream>
```

```
#include <string>
struct Guard {
    std::string n;
    explicit Guard(const std::string& s) : n(s) {}
    ~Guard() { std::cout << '~' << n << ' '; }
};
void f() { Guard g("f"); throw 42; }
int main() {
    try { Guard m("main"); f(); std::cout << "not printed "; }
    catch (int x) { std::cout << "caught " << x; }
    catch (...) { std::cout << "other"; }
    std::cout << '\n';
}
```

Output:

```
~f ~main caught 42
```

Matches. throw 42 leaves f(), so Guard g ("f") is destroyed first (~f). The exception then leaves the try block, so Guard m ("main") is destroyed (~main) before the handler runs. catch (int x) matches and prints caught 42. "not printed" never appears because the statement after f() is skipped when control jumps to the handler.

Alternative Concept and Complexity Analysis

```
ALGORITHM Process()
BEGIN
    p ← NEW array
    TRY  Work(p)
    CATCH ClaimError   : DELETE p ; RETHROW
    CATCH PackageError : DELETE p ; RETHROW
    CATCH ...          : DELETE p ; RETHROW
END      // plus DELETE p on the normal path
```

Measure	delete in every catch	RAII (vector / unique_ptr)
Time	same	same
Code size	one delete per catch block plus one on the normal path	no cleanup code
Leak places when a new error path is added	every new catch block or early return is a possible leak (n + 1 places for n paths)	0, destructors run automatically

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

Task B1 – Implement and Validate the Polymorphic Billing

B1.1 Implementation (billing.h) and test

```
#pragma once
#include <algorithm>
#include <iostream>
#include <map>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;
struct BillingError : runtime_error { using runtime_error::runtime_error; };
struct ClaimError : BillingError { using BillingError::BillingError; };
struct PackageError : BillingError { using BillingError::BillingError; };
struct RecordError : BillingError { using BillingError::BillingError; };

class Bill {
protected:
```

```
    string patient;
    static double chk(double x) { if (x < 0) throw BillingError("negative charge"); return x; }
public:
    explicit Bill(string p) : patient(move(p)) { if (patient.empty()) throw RecordError("missing
patient record"); }
    virtual ~Bill() = default;
    virtual double total() const = 0;
    virtual string type() const { return "Bill"; }
    void print() const { cout << type() << ' ' << patient << ": " << total() << '\n'; }
};
class OutPatient : public Bill {
    double c, t;
public:
    OutPatient(string p, double c_, double t_) : Bill(p), c(chk(c_)), t(chk(t_)) {}
    double total() const override { return c + t; }
    string type() const override { return "OP"; }
};
class InPatient : public Bill {
    int days; double perDay, proc, meds;
public:
    InPatient(string p, int d, double r, double pr, double m) : Bill(p), days(d), perDay(chk(r)),
proc(chk(pr)), meds(chk(m)) {}
    double total() const override { return days * perDay + proc + meds; }
    string type() const override { return "IP"; }
};
class DayCare : public Bill {
    double pkg;
public:
    DayCare(string p, double k) : Bill(p), pkg(chk(k)) {}
    double total() const override { return pkg; }
    string type() const override { return "DC"; }
};
class Emergency : public Bill {
    double base, hrs, rate;
public:
    Emergency(string p, double b, double h, double r) : Bill(p), base(chk(b)), hrs(chk(h)),
rate(chk(r)) {}
    double total() const override { return base + hrs * rate; }
    string type() const override { return "ER"; }
};

class Payer {
public:
    virtual ~Payer() = default;
    virtual double amountCovered(const Bill& b) const = 0;
    virtual string name() const = 0;
};
class SelfPay : public Payer {
public:
    double amountCovered(const Bill&) const override { return 0; }
    string name() const override { return "Self"; }
};
class Insurance : public Payer {
    double limit, copay;
public:
    Insurance(double l, double c) : limit(l), copay(c) {}
    double amountCovered(const Bill& b) const override {
        if (b.total() > limit) throw ClaimError("exceeds cover");
        return b.total() * (100 - copay) / 100;
    }
    string name() const override { return "Insurance"; }
};
class GovScheme : public Payer {
    map<string, double> rate;
public:
    explicit GovScheme(map<string, double> r) : rate(move(r)) {}
    double amountCovered(const Bill& b) const override {
        auto it = rate.find(b.type());
        if (it == rate.end()) throw PackageError("unknown package " + b.type());
        return min(it->second, b.total());
    }
    string name() const override { return "Govt"; }
```

```
};

inline void runTests() {
    vector<unique_ptr<Bill>> bills;
    bills.emplace_back(new OutPatient("Ravi", 500, 1200));
    bills.emplace_back(new InPatient("Meena", 3, 2500, 40000, 2000));
    bills.emplace_back(new DayCare("Kumar", 30000));
    bills.emplace_back(new Emergency("Lata", 1000, 4, 500));
    vector<unique_ptr<Payer>> payers;
    payers.emplace_back(new SelfPay);
    payers.emplace_back(new Insurance(50000, 10));
    payers.emplace_back(new GovScheme({{"OP", 800}, {"IP", 30000}, {"DC", 30000}, {"ER", 2000}}));
    cout << "Bill Payer Total Covered PatientPays\n";
    for (auto& b : bills) for (auto& p : payers) {
        double c = p->amountCovered(*b);
        cout << b->type() << ' ' << p->name() << ' ' << b->total() << ' ' << c << ' ' << b->total()
- c << '\n';
    }
}
```

```
#include "billing.h"
int main() { runTests(); }
```

Output (one bill of each kind with each payer; Insurance limit 50,000 and co-payment 10%; Government rates OP 800, IP 30,000, DC 30,000, ER 2,000):

Output:

```
Bill Payer Total Covered PatientPays
OP Self 1700 0 1700
OP Insurance 1700 1530 170
OP Govt 1700 800 900
IP Self 49500 0 49500
IP Insurance 49500 44550 4950
IP Govt 49500 30000 19500
DC Self 30000 0 30000
DC Insurance 30000 27000 3000
DC Govt 30000 30000 0
ER Self 3000 0 3000
ER Insurance 3000 2700 300
ER Govt 3000 2000 1000
```

Hand verification of the bill totals and of every covered amount:

Bill	Hand calculation	Total	Self	Insurance (×0.9)	Govt (min(rate,total))
OP	500 + 1200	1700	0	1530	min(800,1700) = 800
IP	3×2500 + 40000 + 2000	49500	0	44550	min(30000,49500) = 30000
DC	package	30000	0	27000	min(30000,30000) = 30000
ER	1000 + 4×500	3000	0	2700	min(2000,3000) = 2000

All 12 rows of the program output equal the hand-calculated values (patient pays = total − covered).

B1.2 Comparison with the Part A design

- type() was given a default body ("Bill") instead of being pure virtual, so only total() is pure virtual (the Complete the Code spec).
- Payer gained a name() function for printing, and GovernmentScheme uses type() as the package key.
- A small protected helper Bill::chk() was added to reject negative charges in constructors, and the exception classes were added to the header.

B1.3 Prove runtime binding

With virtual removed (b1v.cpp, Bill::total() non-virtual returning 0, derived classes returning 1700) and bills processed through Bill*:

```
#include <iostream>
struct Bill { double total() const { return 0; } };            // virtual removed
struct OP : Bill { double total() const { return 1700; } };
int main() { Bill* b[] = {new OP, new OP}; double s = 0; for (Bill* x : b) s += x->total();
std::cout << "Total " << s << '\n'; for (Bill* x : b) delete x; }
```

Output:

```
Total 0
```

The base version is always called (early binding), so the total is 0 instead of 3400. If the pure specifier is kept without virtual (double total() const = 0;), g++ refuses to compile:

```
error: initializer specified for non-virtual method 'double Bill::total() const'
error: 'double OutPatient::total() const' marked 'override', but does not override
```

B1.4 Prove the virtual destructor

Removing virtual from ~Bill() and running under -fsanitize=address gives:

```
warning: deleting object of abstract class type 'Bill' which has non-virtual destructor will cause
undefined behavior
ERROR: AddressSanitizer: new-delete-type-mismatch
  size of the allocated type:   56 bytes;
  size of the deallocated type: 40 bytes.
```

The object was allocated as OutPatient (56 bytes) but destroyed as Bill (40 bytes); the derived destructor never ran. With virtual ~Bill() the sanitizer report is clean.

Complete the Code (TODO 1 – 3)

```
// TODO 1 (in Bill)
virtual double total() const = 0;
virtual std::string type() const { return "Bill"; }
// TODO 2 (in OutPatient)
double total() const override { return consultation + tests; }
std::string type() const override { return "OP"; }
// TODO 3 (in InPatient)
double total() const override { return days * perDay + surgery; }
std::string type() const override { return "IP"; }
```

```
#include <iostream>
#include <string>
class Bill {
protected:
    std::string patient;
public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual ~Bill() = default;
    virtual double total() const = 0;
    virtual std::string type() const { return "Bill"; }
    void print() const { std::cout << type() << ' ' << patient << ": " << total() << '\n'; }
};
class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(const std::string& p, double c, double t) : Bill(p), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
    std::string type() const override { return "OP"; }
};
class InPatient : public Bill {
    int days; double perDay, surgery;
public:
    InPatient(const std::string& p, int d, double rate, double s) : Bill(p), days(d), perDay(rate),
surgery(s) {}
```

```
    double total() const override { return days * perDay + surgery; }
    std::string type() const override { return "IP"; }
};
int main() {
    Bill* bills[] = {new OutPatient("Ravi", 500, 1200), new InPatient("Meena", 3, 2500, 40000)};
    double sum = 0;
    for (Bill* b : bills) { b->print(); sum += b->total(); }
    std::cout << "Total " << sum << '\n';
    for (Bill* b : bills) delete b;
}
```

Output:

```
OP Ravi: 1700
IP Meena: 47500
Total 49200
```

Removing virtual from total() gives the compile error: initializer specified for non-virtual method 'double Bill::total() const', because a pure specifier (= 0) is only allowed on virtual functions; the overrides then also fail with 'marked override, but does not override'.

Alternative Concept and Complexity Analysis

Templates (compile-time polymorphism), one container per bill type:

```
#include <iostream>
#include <vector>
struct OP { double c, t; double total() const { return c + t; } };
struct IP { int d; double r, p; double total() const { return d * r + p; } };
template <class T> double sum(const std::vector<T>& v) { double s = 0; for (auto& x : v) s +=
x.total(); return s; }
int main() {
    std::vector<OP> ops{{500, 1200}}; std::vector<IP> ips{{3, 2500, 40000}};
    std::cout << sum(ops) + sum(ips) << ' ' << sizeof(OP) << ' ' << sizeof(IP) << '\n';
}
```

Output:

```
49200 16 24
```

Measure	Virtual functions	Templates, one container per type
Time per bill	O(1) plus a virtual call	O(1), no virtual call (can be inlined)
Space	one container; vptr per object; objects often heap-allocated	one vector per type, no vptr (OP 16 bytes, IP 24 bytes, stored contiguously)
All bill types in one list	Possible (Bill*)	Not possible: a vector holds one type only, so a mixed list needs variant or a common base again, and the code must name every type

Task B2 – Engineer Error Conditions

Error tests (b2.cpp)

```
#include "billing.h"
#include <functional>
struct Res { string n; Res(string s) : n(s) {} ~Res() { cout << "   ~" << n << "\n"; } };
void test(const string& name, function<void()> f) {
    cout << name << ": ";
    Res r(name);
    try { f(); cout << "no error"; }
    catch (const ClaimError& e) { cout << "ClaimError(" << e.what() << ")"; }
    catch (const PackageError& e) { cout << "PackageError(" << e.what() << ")"; }
    catch (const RecordError& e) { cout << "RecordError(" << e.what() << ")"; }
    catch (const BillingError& e) { cout << "BillingError(" << e.what() << ")"; }
    catch (const invalid_argument& e) { cout << "invalid_argument(" << e.what() << ")"; }
    catch (const bad_alloc& e) { cout << "bad_alloc(" << e.what() << ")"; }
```

```
    cout << '\n';
}
int main() {
    test("negative charge", [] { OutPatient("A", -5, 10); });
    test("claim above cover", [] { Insurance(1000, 10).amountCovered(DayCare("A", 30000)); });
    test("unknown package", [] { GovScheme({{"OP", 800}}).amountCovered(DayCare("A", 30000)); });
    test("missing record", [] { OutPatient("", 1, 1); });
    test("text for number", [] { stod("abc"); });
#ifndef NOBAD
    test("allocation failure", [] { volatile size_t n = size_t(-1) / 2; char* p = new char[n];
delete[] p; });
#endif
    try { Res a("A"); Res b("B"); throw 7; }
    catch (...) { cout << "top-level catch(...) reached\n"; }
    cout << "program still running\n";
}
```

Output (plain build; each destructor message is printed on its own line):

Output:

```
negative charge: BillingError(negative charge)
   ~negative charge
claim above cover: ClaimError(exceeds cover)
   ~claim above cover
unknown package: PackageError(unknown package DC)
   ~unknown package
missing record: RecordError(missing patient record)
   ~missing record
text for number: invalid_argument(stod)
   ~text for number
allocation failure: bad_alloc(std::bad_alloc)
   ~allocation failure
   ~B
   ~A
top-level catch(...) reached
program still running
```

Under -fsanitize=address the program exits with status 0 and no sanitizer report. One exception: AddressSanitizer itself aborts when a program asks for an impossibly large block (it reports 'requested allocation size ... exceeds maximum supported size'), so the allocation-failure test is compiled out of the sanitizer build (-DNOBAD) and run in the plain build above, where std::bad_alloc is caught normally.

What each test reveals

- Negative charge, missing record, claim above cover and unknown package are caught by the most specific handler first; the order of catch blocks matters (derived before base).
- std::stod("abc") throws std::invalid_argument, which is outside my BillingError tree, so the design must also handle standard exceptions.
- bad_alloc is a separate failure that is handled in the same way, and the process keeps running.
- catch(...) at the top level keeps the program running; ~B and ~A confirm that unwinding destroyed all objects (reverse order) before the handler ran.

Predict the Output

My predicted output: ok done caught negative [log too large] caught too large

```
#include <iostream>
#include <stdexcept>
void validate(double amount) {
    if (amount < 0) throw std::invalid_argument("negative");
    if (amount > 1e6) throw std::out_of_range("too large");
    std::cout << "ok ";
}
void process(double amount) {
    try { validate(amount); }
    catch (const std::out_of_range& e) { std::cout << "[log " << e.what() << "] "; throw; }
}
int main() {
```

```
    for (double a : {100.0, -5.0, 2e6}) {
        try { process(a); std::cout << "done "; }
        catch (const std::exception& e) { std::cout << "caught " << e.what() << ' '; }
    }
    std::cout << '\n';
}
```

Output:

```
ok done caught negative [log too large] caught too large
```

Matches. 100: validate prints ok, then done. -5: validate throws invalid_argument; process catches only out_of_range, so it passes through to main, whose catch(const std::exception&) prints caught negative. 2e6: validate throws out_of_range; process matches, logs [log too large] and rethrows with throw;, and main prints caught too large. Only one exception is logged because only out_of_range has a handler in process().

Complete the Code (TODO 1 and 2)

```
// TODO 1 (in approve)
if (billed < 0 || cover < 0) throw std::invalid_argument("negative amount");
if (billed > cover) throw ClaimError("exceeds cover");
return billed;
// TODO 2 (in settle's catch block)
std::cout << "audit: " << e.what() << '\n'; throw;
```

```
#include <iostream>
#include <stdexcept>
class ClaimError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};
double approve(double billed, double cover) {
    if (billed < 0 || cover < 0) throw std::invalid_argument("negative amount");
    if (billed > cover) throw ClaimError("exceeds cover");
    return billed;
}
double settle(double billed, double cover) {
    try { return approve(billed, cover); }
    catch (const ClaimError& e) { std::cout << "audit: " << e.what() << '\n'; throw; }
}
int main() {
    double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
    for (auto& c : cases) {
        try { double paid = settle(c[0], c[1]); std::cout << "approved " << paid << '\n'; }
        catch (const ClaimError& e) { std::cout << "claim rejected: " << e.what() << '\n'; }
        catch (const std::exception& e) { std::cout << "error: " << e.what() << '\n'; }
    }
}
```

Output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```

Replacing throw; by throw e;: throw; rethrows the original exception object (same dynamic type, no copy). throw e; throws a new copy of e whose type is the static type of e. In this program e is declared const ClaimError&, so the output is identical. The difference appears when the handler catches a base class: below, e is std::runtime_error&, so throw e; slices the object to std::runtime_error and the later ClaimError handler no longer matches.

```
#include <iostream>
#include <stdexcept>
struct ClaimError : std::runtime_error { using std::runtime_error::runtime_error; };
void f(bool copy) {
    try { throw ClaimError("exceeds cover"); }
```

```
    catch (const std::runtime_error& e) { if (copy) throw e; else throw; }   // caught by base
reference
}
int main() {
    for (bool c : {false, true})
        try { f(c); } catch (const ClaimError& e) { std::cout << "ClaimError handler\n"; }
                      catch (const std::exception& e) { std::cout << "sliced to base: generic
handler\n"; }
}
```

Output:

```
ClaimError handler
sliced to base: generic handler
```

First line is throw; (ClaimError preserved), second line is throw e; (sliced).

Alternative Concept and Complexity Analysis

```
ALGORITHM Approve(billed, cover) RETURNS optional
BEGIN
    IF billed < 0 OR billed > cover THEN RETURN empty
    RETURN billed
END
caller: r ← Approve(...) ; IF r is empty THEN reject (the reason is lost)
```

1,000,000 claims (measured)	optional<double>	exceptions
0% invalid	0.60 ms	2.11 ms
1% invalid	0.60 ms	8.64 ms
50% invalid	4.51 ms	339.80 ms
Extra space per result	16 bytes (8 value + 1 flag, padded)	none for the result (8 bytes), exception object only when thrown
Reason for rejection	not carried (only 'no value')	carried in the exception type and message

Task B3 – Virtual Functions vs switch vs Exceptions

Benchmark program (bench.cpp, compiled with -O2)

```
#include <chrono>
#include <cstdio>
#include <memory>
#include <optional>
#include <random>
#include <stdexcept>
#include <variant>
#include <vector>
using namespace std;
using Clock = chrono::steady_clock;
template <class F> double ms(F f) { double best = 1e18; for (int r = 0; r < 3; ++r) { auto t =
Clock::now(); f(); best = min(best, chrono::duration<double, milli>(Clock::now() - t).count()); }
return best; }
volatile double sink;

// ---- (a) virtual ----
struct VB { virtual ~VB() = default; virtual double total() const = 0; };
struct VOP : VB { double c, t; VOP(double a, double b) : c(a), t(b) {} double total() const override
{ return c + t; } };
struct VIP : VB { int d; double r, p; VIP(int a, double b, double c) : d(a), r(b), p(c) {} double
total() const override { return d * r + p; } };
struct VDC : VB { double k; VDC(double a) : k(a) {} double total() const override { return k; } };
struct VER : VB { double b, h, r; VER(double x, double y, double z) : b(x), h(y), r(z) {} double
total() const override { return b + h * r; } };
// ---- (b) switch ----
struct SB { int type; double a, b, c; };
```

```
double stotal(const SB& s) { switch (s.type) { case 0: return s.a + s.b; case 1: return s.a * s.b +
s.c; case 2: return s.a; default: return s.a + s.b * s.c; } }
// ---- (c) variant ----
struct XOP { double c, t; double total() const { return c + t; } };
struct XIP { int d; double r, p; double total() const { return d * r + p; } };
struct XDC { double k; double total() const { return k; } };
struct XER { double b, h, r; double total() const { return b + h * r; } };
using Var = variant<XOP, XIP, XDC, XER>;
// ---- (d) rule table (data driven plug-in) ----
struct Rule { double k0, k1, k2; bool mulFirst; };
double rtotal(const Rule& r, const SB& s) { return r.k0 * s.a + r.k1 * s.b + r.k2 * s.c +
(r.mulFirst ? 0 : 0); }
// ---- (e) dynamic_cast chain ----
double dtotal(const VB* b) { if (auto p = dynamic_cast<const VOP*>(b)) return p->total(); if (auto p
= dynamic_cast<const VIP*>(b)) return p->total();
    if (auto p = dynamic_cast<const VDC*>(b)) return p->total(); if (auto p = dynamic_cast<const
VER*>(b)) return p->total(); return 0; }

int main() {
    mt19937 g(1); uniform_int_distribution<int> ty(0, 3); uniform_real_distribution<double> v(1,
100);
    // B3.1 correctness on 10 bills
    { double sv = 0, ss = 0; vector<unique_ptr<VB>> a; vector<SB> b;
      for (int i = 0; i < 10; ++i) { int t = i % 4; double x = 10 + i, y = 2 + i, z = 3 + i;
        if (t == 0) a.emplace_back(new VOP(x, y)); else if (t == 1) a.emplace_back(new VIP(int(y),
x, z)); else if (t == 2) a.emplace_back(new VDC(x)); else a.emplace_back(new VER(x, y, z));
        b.push_back(t == 1 ? SB{1, y, x, z} : SB{t, x, y, z}); }
      for (auto& p : a) sv += p->total(); for (auto& s : b) ss += stotal(s);
      printf("B3.1 virtual=%.2f switch=%.2f %s\n", sv, ss, sv == ss ? "AGREE" : "DIFFER"); }
    printf("sizeof: VOP=%zu VIP=%zu VER=%zu SB=%zu Var=%zu\n", sizeof(VOP), sizeof(VIP),
sizeof(VER), sizeof(SB), sizeof(Var));
    printf("B3.2 time in ms (O2, best of 3)\n n  virtual  switch  variant  rules  dyn_cast\n");
    for (size_t n : {1000ul, 1000000ul, 10000000ul}) {
        vector<unique_ptr<VB>> a; a.reserve(n); vector<SB> b(n); vector<Var> c; c.reserve(n);
        for (size_t i = 0; i < n; ++i) { int t = ty(g); double x = v(g), y = v(g), z = v(g); int d =
1 + int(y) % 9;
            switch (t) { case 0: a.emplace_back(new VOP(x, y)); b[i] = {0, x, y, 0};
c.emplace_back(XOP{x, y}); break;
                         case 1: a.emplace_back(new VIP(d, x, z)); b[i] = {1, d, x, z};
c.emplace_back(XIP{d, x, z}); break;
                         case 2: a.emplace_back(new VDC(x)); b[i] = {2, x, 0, 0};
c.emplace_back(XDC{x}); break;
                         default: a.emplace_back(new VER(x, y, z)); b[i] = {3, x, y, z};
c.emplace_back(XER{x, y, z}); } }
        Rule rule{1, 1, 1, true};
        double t1 = ms([&] { double s = 0; for (auto& p : a) s += p->total(); sink = s; });
        double t2 = ms([&] { double s = 0; for (auto& x : b) s += stotal(x); sink = s; });
        double t3 = ms([&] { double s = 0; for (auto& x : c) s += visit([](auto& q) { return
q.total(); }, x); sink = s; });
        double t4 = ms([&] { double s = 0; for (auto& x : b) s += rtotal(rule, x); sink = s; });
        double t5 = ms([&] { double s = 0; for (auto& p : a) s += dtotal(p.get()); sink = s; });
        printf("%9zu %8.3f %7.3f %8.3f %6.3f %8.3f\n", n, t1, t2, t3, t4, t5);
    }
    // B3.3 / B2-alt: error reporting cost for 1,000,000 bills
    printf("B3.3 time in ms for 1,000,000 bills\n invalid%%  return_code  optional  exception\n");
    const size_t N = 1000000; vector<double> amt(N);
    for (double pct : {0.0, 1.0, 50.0}) {
        uniform_real_distribution<double> u(0, 100); for (auto& x : amt) x = u(g) < pct ? -1 : 100;
        double ok = 0;
        double r1 = ms([&] { double s = 0; for (double x : amt) { int st = x < 0 ? 1 : 0; if (st)
{ s += 0; continue; } s += x; } sink = s; });
        double r2 = ms([&] { double s = 0; for (double x : amt) { optional<double> o; if (x >= 0) o
= x; if (o) s += *o; } sink = s; });
        double r3 = ms([&] { double s = 0; for (double x : amt) { try { if (x < 0) throw
invalid_argument("neg"); s += x; } catch (const invalid_argument&) {} } sink = s; });
        (void)ok; printf("%8.0f %12.3f %9.3f %10.3f\n", pct, r1, r2, r3);
    }
}
```

B3.1 Correctness on 10 bills

The same 10 bills were totalled with virtual functions and with a single struct plus switch: virtual = 609.00, switch = 609.00, AGREE.

B3.2 Increasing the data size (milliseconds, best of 3)

Bills	virtual	switch	variant + visit	rule table	dynamic_cast chain
1,000	0.005	0.004	0.002	0.001	0.027
1,000,000	9.23	8.01	7.12	1.92	29.15
10,000,000	95.27	87.83	74.23	28.88	306.05

B3.3 Cost of exceptions (1,000,000 bills, milliseconds)

Invalid bills	return code	optional	exception
0%	0.61	0.60	2.11
1%	0.57	0.60	8.64
50%	3.92	4.51	339.80

B3.4 Trade-off

- Virtual and switch are both O(n); the virtual version is about 8% slower at 10 million bills (95.3 vs 87.8 ms). The cost is one indirect call and poorer cache use (each bill is a separate heap object, so reads are less sequential), which is small compared with the benefit of adding a type without editing existing code.
- Exceptions cost almost nothing when no error occurs, but each throw is expensive: about 0.8 µs per throw at 1% invalid bills and about 0.68 µs per throw at 50%, versus about 0.007 µs per error for return codes. Use exceptions for rare, exceptional failures (invalid claim, bad record); use return codes or optional where bad input is common (for example validating a file with 50% bad rows in a tight loop).
- Use a switch only for a fixed, closed set of types in performance-critical code; use virtual functions when new types are added regularly.

Alternative Concept and Complexity Analysis

std::variant<OP, IP, DC, ER> with std::visit (columns above): at 10,000,000 bills it took 74.2 ms against 87.8 ms for the switch and 95.3 ms for virtual calls, because the objects are stored contiguously and no per-object heap allocation is needed.

Measure	variant + visit	virtual	switch
Time, 10,000,000 bills	74.2 ms	95.3 ms	87.8 ms
Space per element (measured)	32 bytes (largest alternative 24 + tag, padded)	24–32 bytes per object (includes the vptr) + 8-byte pointer + heap overhead	32 bytes (all fields)
Adding a new type	edit the variant type list (existing classes unchanged, but a recompile of users)	new class only	edit struct and switch

Task B4 – Final Engineering Decision

B4.1 Practical extension (b4.cpp)

A new bill type TeleConsult and a new payer CorporateTieUp are added in a separate file; billing.h is not edited.

```
#include "billing.h"
// ---- new classes only; billing.h is untouched ----
class TeleConsult : public Bill {
    double fee;
public:
    TeleConsult(string p, double f) : Bill(p), fee(chk(f)) {}
    double total() const override { return fee; }
    string type() const override { return "TC"; }
};
class Corporate : public Payer {
    double cap;
public:
    explicit Corporate(double c) : cap(c) {}
    double amountCovered(const Bill& b) const override { return min(cap, b.total()); }
    string name() const override { return "Corporate"; }
};
int main() {
    runTests();                                    // all earlier tests (same output as b1)
    cout << "--- extension ---\n";
    TeleConsult tc("Anu", 600); OutPatient op("Ravi", 500, 1200); InPatient ip("Meena", 3, 2500,
40000, 2000);
    Corporate corp(5000);
    for (const Bill* b : {(const Bill*)&tc, (const Bill*)&op, (const Bill*)&ip})
        cout << b->type() << " Corporate " << b->total() << ' ' << corp.amountCovered(*b) << '\n';
    cout << "--- errors ---\n";
    GovScheme gov({{"OP", 800}});
    try { gov.amountCovered(tc); } catch (const PackageError& e) { cout << "PackageError: " <<
e.what() << '\n'; }
    try { Insurance(1000, 10).amountCovered(ip); } catch (const ClaimError& e) { cout <<
"ClaimError: " << e.what() << '\n'; }
    try { TeleConsult("Bad", -1); } catch (const BillingError& e) { cout << "BillingError: " <<
e.what() << '\n'; }
}
```

Output:

```
Bill Payer Total Covered PatientPays
OP Self 1700 0 1700
OP Insurance 1700 1530 170
OP Govt 1700 800 900
IP Self 49500 0 49500
IP Insurance 49500 44550 4950
IP Govt 49500 30000 19500
DC Self 30000 0 30000
DC Insurance 30000 27000 3000
DC Govt 30000 30000 0
ER Self 3000 0 3000
ER Insurance 3000 2700 300
ER Govt 3000 2000 1000
--- extension ---
TC Corporate 600 600
OP Corporate 1700 1700
IP Corporate 49500 5000
--- errors ---
PackageError: unknown package TC
ClaimError: exceeds cover
BillingError: negative charge
```

The first 13 lines are byte-for-byte identical to the output of b1 (checked with diff), so all earlier tests still pass. The extension lines show that the new bill type works with the new payer, and the error lines show that the Government scheme rejects the unknown package TC (PackageError).

B4.2 Resource safety

b4.cpp was compiled with g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined and run: exit status 0 and no output on standard error (no leak, no invalid access) even though PackageError, ClaimError and BillingError were thrown and caught.

```
$ g++ -std=c++17 -Wall -Wextra -fsanitize=address,undefined -g b4.cpp -o b4
$ ./b4 > b4.out 2> b4.err ; echo $?
0            (b4.err is empty: 0 bytes)
```

B4.3 Production decision

Decision: adopt the polymorphic design with exceptions for the hospital group.

- Volume: 200,000 bills a day is tiny for this design. Totalling 10,000,000 bills took about 95 ms, so a day's bills take roughly 2 ms.
- Change rate: two new bill types and three new payers a year means about five additions a year. Each is a new class and a few lines in a factory, with no edit to tested code (shown in B4.1), so risk and effort stay low; a switch would force edits and re-testing of every switch each time.
- Errors: invalid claims are rare, so the 0.7 µs cost per exception is irrelevant (even 50% invalid bills would cost under a second per million). Exceptions carry the reason and keep the billing code clear.
- Safety: RAII containers, virtual destructors and a clean sanitizer report show that no bill is lost or leaked on errors.
- Keep in mind: if rule changes become very frequent or are edited by non-programmers, move the rates to data files (see below) while keeping the polymorphic classes for the logic.

Alternative Concept and Complexity Analysis

```
ALGORITHM LoadRules(file)                // plug-in: rates described in a configuration file
BEGIN
    FOR EACH line IN file DO  rules[type] ← (k0, k1, k2)
END
ALGORITHM Total(bill)
BEGIN
    r ← rules[bill.type]
    RETURN r.k0*bill.a + r.k1*bill.b + r.k2*bill.c      // interpreted rule
END
```

Measure	New C++ classes (virtual)	Configuration rules
Time per bill	O(1), one virtual call (about 9.5 ns per bill measured)	O(1) with a table lookup, but O(r) if a rule has r terms or must be parsed; my simple 3-term rule took 2.9 ns per bill, a real rule language would be slower
Space for rules	code in the program; no data tables	table of rules in memory, one record per type
Effort to extend	write, compile and test a class; needs a programmer and a release	edit a text file; no recompile
Risk	Compiler checks types; mistakes are found at build time; tests protect old classes	Wrong rate or syntax is found only at run time; limited to what the rule format can express, so a new kind of calculation still needs code